

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN
KHOA KHOA HỌC VÀ KỸ THUẬT THÔNG TIN



ĐỒ ÁN MÔN HỌC: DỮ LIỆU LỚN (BIG DATA)

HỆ THỐNG GỢI Ý SẢN PHẨM SOFTWARE TRÊN AMAZON

*Kiến trúc hai tháp (Two-Tower) với mô hình ngôn ngữ
và học tương phản trên nền tảng dữ liệu lớn*

Giảng viên hướng dẫn: Phạm Huy Thanh

Sinh viên thực hiện: Đào Minh Trí

Mã số sinh viên:

Lớp:

TP. Hồ Chí Minh, tháng 6 năm 2026

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy **Phạm Huy Thanh**, giảng viên hướng dẫn môn *Dữ liệu lớn (Big Data)*. Thầy đã tận tình truyền đạt những kiến thức nền tảng về hệ sinh thái xử lý dữ liệu phân tán, đồng thời định hướng và góp ý chi tiết trong suốt quá trình em thực hiện đồ án này. Những phản hồi quý báu của thầy giúp em hiểu rõ hơn mối liên hệ giữa lý thuyết hệ thống gợi ý và bài toán xử lý dữ liệu quy mô lớn trong thực tế.

Em cũng xin cảm ơn Trường Đại học Công nghệ Thông tin - Đại học Quốc gia Thành phố Hồ Chí Minh và Khoa Khoa học và Kỹ thuật Thông tin đã tạo điều kiện về môi trường học tập, tài nguyên tính toán và nguồn học liệu phong phú để em có thể hoàn thành đồ án.

Do kiến thức và kinh nghiệm còn hạn chế, đồ án khó tránh khỏi những thiếu sót. Em rất mong nhận được sự góp ý của thầy và các bạn để báo cáo được hoàn thiện hơn.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 6 năm 2026

Sinh viên thực hiện

Đào Minh Trí

TÓM TẮT ĐỒ ÁN

Báo cáo trình bày quá trình thiết kế và xây dựng một **hệ thống gợi ý (Recommender System)** cho nhóm sản phẩm *Software* trên sàn thương mại điện tử Amazon, đặt trong bối cảnh **dữ liệu lớn (Big Data)**. Khác với các hệ thống lọc cộng tác truyền thống dựa trên định danh – vốn bỏ qua văn bản ngữ nghĩa và gặp khó với bài toán khởi động lạnh – đồ án đề xuất một hướng tiếp cận hiện đại kết hợp **mô hình ngôn ngữ, kiến trúc hai tháp (Two-Tower)** và **học tương phản (contrastive learning)**.

Toàn bộ pipeline tiền xử lý được triển khai trên **Apache Spark (PySpark)**: nạp dữ liệu Amazon Reviews thô, làm sạch, ghép nối các tập chia theo thời gian và dựng văn bản ngữ nghĩa (*semantic_text*) cho hơn 17.000 sản phẩm, lưu trữ tối ưu bằng định dạng cột **Parquet**. Mỗi sản phẩm được mã hóa thành vector 384 chiều bằng mô hình *all-MiniLM-L6-v2* (đóng băng); vector lịch sử của 146.000 người dùng được tổng hợp với **trọng số thời gian** kết hợp độ gần đây và mức độ ưa thích. Mô hình hai tháp được huấn luyện bằng hàm mất mát **InfoNCE** với mẫu âm trong batch, tối ưu bằng AdamW và tính toán độ chính xác hỗn hợp trên GPU.

Hệ thống được đánh giá bằng giao thức **toàn kho (full-catalog)** khắt khe – mã hóa trước toàn bộ kho sản phẩm rồi để mỗi người dùng cạnh tranh trên toàn bộ kho. Kết quả cho thấy mô hình đạt $\text{Recall}@10 = 0,0142$ và $\text{Recall}@50 = 0,0436$, **vượt trội hơn 24 lần ở Top-10 và hơn 14 lần ở Top-50** so với baseline chọn ngẫu nhiên, đồng thời hội tụ ổn định mà không quá khớp. Báo cáo kết luận bằng việc chỉ ra hạn chế của tháp sản phẩm đóng băng và đề xuất hướng phát triển: tinh chỉnh tháp sản phẩm bằng LoRA và tích hợp tìm kiếm vector siêu tốc FAISS cho gợi ý thời gian thực.

Từ khóa: *Hệ thống gợi ý, Two-Tower, Mô hình ngôn ngữ, Sentence Embedding, Học tương phản, InfoNCE, Apache Spark, Big Data, Amazon Reviews.*

MỤC LỤC

1	Tổng quan đề tài	1
1.1	Đặt vấn đề	1
1.2	Giải pháp đề xuất	1
1.3	Mục tiêu và phạm vi	2
1.4	Đóng góp của báo cáo	2
1.5	Cấu trúc báo cáo	3
2	Cơ sở lý thuyết	4
2.1	Hệ thống gợi ý theo hướng truy hồi	4
2.2	Các hướng nghiên cứu liên quan	4
2.3	Nền tảng dữ liệu lớn	5
2.3.1	Apache Spark	5
2.3.2	Định dạng cột Apache Parquet	5
2.4	Mô hình ngôn ngữ và biểu diễn văn bản	6
2.4.1	Kiến trúc Transformer	6
2.4.2	Từ văn bản tới vector ngữ nghĩa	6
2.4.3	Tiền huấn luyện và Sentence-BERT	7
2.5	Kiến trúc hai tháp (Two-Tower)	7
2.6	Học tương phản và hàm mất mát InfoNCE	8
2.6.1	Vai trò của tham số nhiệt độ	9
2.7	Tìm kiếm láng giềng gần nhất	9
2.8	Các độ đo đánh giá	10
2.8.1	Toàn kho và thiên lệch lấy mẫu	10
3	Phân tích và thiết kế hệ thống	12
3.1	Kiến trúc tổng thể	12
3.2	Dữ liệu và tiền xử lý bằng PySpark	13
3.2.1	Làm sạch và kiểm tra chất lượng dữ liệu	14
3.2.2	Dựng văn bản ngữ nghĩa cho sản phẩm	14

3.3	Chiến lược chia dữ liệu theo thời gian	16
3.4	Mã hóa trải nghiệm và tổng hợp lịch sử người dùng	16
3.4.1	Mã hóa trải nghiệm	16
3.4.2	Tổng hợp lịch sử với trọng số thời gian	17
3.5	Thiết kế mô hình hai tháp	19
3.5.1	Sơ đồ kiến trúc chi tiết	19
3.5.2	Phân tích số tham số	20
3.6	Hàm mất mát và cấu hình huấn luyện	21
4	Kết quả thực nghiệm	22
4.1	Môi trường và cấu hình	22
4.2	Chi tiết vòng lặp huấn luyện	23
4.3	Thiết kế nạp dữ liệu theo batch	24
4.4	Giao thức đánh giá toàn kho (Full-catalog)	24
4.5	Kết quả huấn luyện	25
4.6	Kết quả truy hồi trên toàn kho	26
4.6.1	So sánh với baseline ngẫu nhiên	26
4.7	Ví dụ gợi ý định tính	27
4.8	Phân tích không gian biểu diễn	28
4.9	Phân tích và thảo luận	29
5	Kết luận và hướng phát triển	30
5.1	Kết luận	30
5.2	Hạn chế	30
5.3	Hướng phát triển	31
5.3.1	Phân tích hai hướng nâng cấp trọng tâm	31
	Tài liệu tham khảo	32
A	Hướng dẫn cài đặt và tái lập kết quả	34
B	Tổng hợp siêu tham số và kết quả đầy đủ	35

DANH SÁCH HÌNH VẼ

2.1	Cấu trúc một tầng mã hóa Transformer (MiniLM xếp chồng 6 tầng)	6
2.2	Kiến trúc hai tháp (Two-Tower) với tháp sản phẩm đóng băng	8
2.3	Ma trận tương đồng in-batch của InfoNCE; đường chéo là cặp dương cần tối đa hóa	9
2.4	Recall@10 giảm khi số ứng viên đánh giá tăng – minh chứng thiên lệch lấy mẫu	11
3.1	Sơ đồ khối end-to-end: từ xử lý dữ liệu đến mô hình và đánh giá	12
3.2	Phân phối số tương tác trên mỗi người dùng	13
3.3	Một số đặc trưng của dữ liệu Amazon Software	16
3.4	Chia dữ liệu leave-last-out: mục cuối làm test, kế cuối làm val, còn lại train	16
3.5	Trọng số thời gian khi tổng hợp vector lịch sử người dùng ($n = 15$)	18
3.6	Luồng tổng hợp vector lịch sử người dùng từ các vector trải nghiệm . . .	19
3.7	Sơ đồ kiến trúc chi tiết của mô hình hai tháp kèm kích thước tensor	20
3.8	Phân bổ tham số giữa hai tháp (chỉ $\sim 1,7\%$ được huấn luyện)	21
4.1	Đường cong mất mát InfoNCE trên tập huấn luyện và kiểm định	25
4.2	Recall@K (trái) và NDCG@K (phải) trên tập kiểm định qua các epoch . .	26
4.3	Hiệu năng truy hồi của mô hình so với baseline chọn ngẫu nhiên	27
4.4	Minh họa khái niệm không gian biểu diễn 2 chiều của sản phẩm và người dùng	28

DANH SÁCH BẢNG

3.1	Thống kê quy mô bộ dữ liệu Amazon Software	13
3.2	Số tham số khả huấn luyện của tháp người dùng	20
4.1	Cấu hình môi trường và siêu tham số huấn luyện	22
4.2	Phân rã thời gian một epoch (tốc độ $\sim 1,7$ batch/giây)	23
4.3	Kết quả truy hồi toàn kho theo từng epoch (tập kiểm định)	26
4.4	Bội số cải thiện của mô hình so với chọn ngẫu nhiên	27
4.5	Ví dụ Top-5 gợi ý dựa trên độ tương đồng vector	27
B.1	Tổng hợp siêu tham số của mô hình hai tháp	35
B.2	Kết quả đầy đủ Recall@K và NDCG@K (tập kiểm định, toàn kho)	35

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Thuật ngữ đầy đủ	Ý nghĩa
RS	Recommender System	Hệ thống gợi ý
CF	Collaborative Filtering	Lọc cộng tác
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
MLP	Multi-Layer Perceptron	Mạng perceptron đa tầng
BN	Batch Normalization	Chuẩn hóa theo batch
InfoNCE	Info Noise-Contrastive Estimation	Hàm mất mát tương phản dùng trong đồ án
NCE	Noise-Contrastive Estimation	Ước lượng tương phản nhiễu
NDCG	Normalized Discounted Cumulative Gain	Độ lợi tích lũy chiết khấu chuẩn hóa
AMP	Automatic Mixed Precision	Tính toán độ chính xác hỗn hợp
FP16	Half Precision Floating Point	Số thực dấu phẩy động 16-bit
LoRA	Low-Rank Adaptation	Tinh chỉnh tham số hạng thấp
FAISS	Facebook AI Similarity Search	Thư viện tìm kiếm vector tương đồng
EDA	Exploratory Data Analysis	Phân tích khám phá dữ liệu
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
API	Application Programming Interface	Giao diện lập trình ứng dụng
ASIN	Amazon Standard Identification Number	Mã định danh sản phẩm trên Amazon

Chương 1

Tổng quan đề tài

1.1 Đặt vấn đề

Thương mại điện tử hiện đại đặt người dùng trước hàng triệu lựa chọn, dẫn tới hiện tượng *quá tải thông tin* (information overload). **Hệ thống gợi ý** (Recommender System) ra đời nhằm tự động lọc và đề xuất cho mỗi người dùng những sản phẩm phù hợp nhất với sở thích của họ. Đối với nhóm sản phẩm **Software** (phần mềm) trên Amazon, bài toán càng quan trọng bởi sản phẩm mang tính chuyên biệt cao, mô tả kỹ thuật dài và đánh giá người dùng giàu ngữ nghĩa.

Tuy nhiên, các hệ thống gợi ý truyền thống dựa trên **định danh** (ID-based collaborative filtering) bộc lộ hai hạn chế cố hữu:

- **Bỏ qua dữ liệu văn bản ngữ nghĩa:** các mô hình lọc cộng tác cổ điển chỉ học từ ma trận tương tác (ai đã mua/đánh giá gì), hoàn toàn không khai thác phần mô tả sản phẩm, tính năng hay nội dung review – vốn chứa thông tin rất giá trị về bản chất sản phẩm.
- **Bài toán khởi động lạnh (cold-start):** khi xuất hiện người dùng mới hoặc sản phẩm mới chưa có lịch sử tương tác, mô hình dựa trên định danh không thể đưa ra dự đoán đáng tin cậy.

1.2 Giải pháp đề xuất

Đề án đề xuất một hướng tiếp cận hiện đại nhằm khắc phục đồng thời hai hạn chế trên, dựa trên ba trụ cột:

1. **Mô hình ngôn ngữ (Language Model):** sử dụng một mô hình ngôn ngữ dựa trên kiến trúc Transformer để *hiểu* văn bản mô tả sản phẩm và nội dung đánh giá, chuyển toàn bộ thông tin văn bản dài thành một *vector số cố định* đại diện cho ý nghĩa của sản phẩm. Nhờ vậy, hệ thống nắm bắt được sự tương đồng ngữ nghĩa ngay cả với sản phẩm mới.
2. **Kiến trúc hai tháp (Two-Tower):** tách riêng việc biểu diễn *người dùng* và *sản phẩm* thành hai nhánh độc lập, cùng ánh xạ vào một không gian vector chung.

Nhánh người dùng học từ *chuỗi hành vi theo thời gian*, có tính tới yếu tố thời điểm và mức độ ưa thích.

3. **Học tương phản (Contrastive Learning):** huấn luyện mô hình bằng cách kéo các sản phẩm mà người dùng thích lại gần vector người dùng và đẩy các sản phẩm không liên quan ra xa trong không gian biểu diễn.

1.3 Mục tiêu và phạm vi

Mục tiêu của đề án là xây dựng một *luồng xử lý hoàn chỉnh* (end-to-end pipeline) cho hệ thống gợi ý phần mềm, bao gồm:

- Nạp và tiền xử lý dữ liệu lớn bằng **Apache Spark**, lưu trữ tối ưu theo định dạng cột **Apache Parquet**.
- Mã hóa "trải nghiệm" của người dùng thành vector bằng mô hình ngôn ngữ, tích hợp trọng số thời gian.
- Huấn luyện mô hình hai tháp với hàm mất mát tương phản.
- Đánh giá khách quan độ hiệu quả trên **toàn bộ kho sản phẩm** (full-catalog), thay vì chỉ trên một nhóm nhỏ.

Phạm vi: Báo cáo tập trung vào danh mục *Software* của bộ dữ liệu Amazon Reviews 2023, với khoảng 146.000 người dùng và hơn 17.000 sản phẩm phần mềm độc nhất. Bài toán được mô hình hóa theo dạng **truy hồi** (retrieval): với mỗi người dùng, hệ thống xếp hạng toàn bộ kho sản phẩm và trả về danh sách Top- N phù hợp nhất.

1.4 Đóng góp của báo cáo

- Xây dựng pipeline xử lý dữ liệu lớn end-to-end trên PySpark, tạo ra biểu diễn văn bản ngữ nghĩa (`semantic_text`) thống nhất cho sản phẩm.
- Thiết kế thuật toán tổng hợp vector lịch sử người dùng có *trọng số thời gian* (recency) kết hợp mức độ ưa thích (rating), tính toán hiệu quả trên bộ nhớ.
- Cài đặt và huấn luyện mô hình Two-Tower với mô hình ngôn ngữ làm bộ mã hóa sản phẩm và hàm mất mát tương phản InfoNCE.
- Đề xuất giao thức đánh giá **full-catalog** loại bỏ thiên lệch đánh giá cục bộ, chứng minh mô hình vượt trội nhiều lần so với baseline ngẫu nhiên.

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức theo năm chương:

- **Chương 1 – Tổng quan đề tài:** đặt vấn đề, giải pháp, mục tiêu và phạm vi.
- **Chương 2 – Cơ sở lý thuyết:** nền tảng dữ liệu lớn, mô hình ngôn ngữ, kiến trúc hai tháp và học tương phản.
- **Chương 3 – Phân tích và thiết kế hệ thống:** kiến trúc tổng thể, xử lý dữ liệu, chiến lược chia dữ liệu và thiết kế mô hình.
- **Chương 4 – Kết quả thực nghiệm:** môi trường, giao thức đánh giá và phân tích kết quả.
- **Chương 5 – Kết luận và hướng phát triển:** tổng kết, hạn chế và định hướng tương lai.

Chương 2

Cơ sở lý thuyết

2.1 Hệ thống gợi ý theo hướng truy hồi

Gọi tập người dùng là U và tập sản phẩm là I . Thay vì dự đoán điểm số cho từng cặp như lọc cộng tác cổ điển, đồ án mô hình hóa bài toán theo hướng **truy hồi dựa trên biểu diễn** (embedding-based retrieval): mỗi người dùng u và mỗi sản phẩm i được ánh xạ thành các vector $\mathbf{u}, \mathbf{v} \in \mathbb{R}^d$ trong cùng một không gian. Mức độ phù hợp được đo bằng độ tương đồng cosine:

$$\text{score}(u, i) = \cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}. \quad (2.1)$$

Khi cả hai vector đã được chuẩn hóa L_2 (độ dài bằng 1), độ tương đồng cosine trở thành tích vô hướng đơn giản. Gợi ý Top- N chính là việc tìm N sản phẩm có score cao nhất với người dùng – một bài toán *tìm kiếm láng giềng gần nhất* (nearest neighbor search) trong không gian vector, có thể tăng tốc bằng các thư viện chuyên dụng. Hướng tiếp cận này là nền tảng của các hệ thống gợi ý công nghiệp quy mô lớn nhờ khả năng tách biệt việc tính toán biểu diễn người dùng và sản phẩm.

2.2 Các hướng nghiên cứu liên quan

Lĩnh vực hệ thống gợi ý đã trải qua nhiều giai đoạn phát triển:

- **Lọc cộng tác cổ điển:** từ các phương pháp dựa trên láng giềng (user/item-based KNN) tới *phân rã ma trận* (Matrix Factorization, ALS), học các nhân tố ẩn từ ma trận tương tác. Hạn chế chung là chỉ dùng tín hiệu định danh và gặp khó với cold-start.
- **Học sâu cho gợi ý:** các mô hình như Neural Collaborative Filtering thay tích vô hướng bằng mạng nơ-ron để học tương tác phi tuyến, song vẫn dựa trên embedding theo định danh.
- **Truy hồi hai tháp:** được phổ biến bởi các hệ thống công nghiệp quy mô lớn (YouTube, Google), kiến trúc hai tháp cho phép truy hồi ứng viên cực nhanh từ kho hàng triệu mục, thường huấn luyện bằng học tương phản với mẫu âm trong batch.

- **Gợi ý dựa trên ngôn ngữ:** xu hướng gần đây kết hợp mô hình ngôn ngữ để khai thác văn bản mô tả sản phẩm, giúp biểu diễn giàu ngữ nghĩa và giảm phụ thuộc vào lịch sử tương tác – đặc biệt hữu ích cho cold-start.

Đề án kế thừa và kết hợp hai hướng cuối: dùng *mô hình ngôn ngữ* làm tháp sản phẩm trong kiến trúc *hai tháp*, huấn luyện bằng *học tương phản* – một thiết kế hiện đại, phù hợp với bối cảnh dữ liệu lớn và đặc thù giàu văn bản của sản phẩm phần mềm.

2.3 Nền tảng dữ liệu lớn

2.3.1 Apache Spark

Apache Spark là nền tảng tính toán phân tán cho phép xử lý dữ liệu quy mô lớn theo cơ chế song song trong bộ nhớ. Trong đề án, Spark đảm nhận toàn bộ khâu tiền xử lý nặng: nạp dữ liệu JSON/JSONL thô, khử trùng lặp, ghép nối (join) các tập dữ liệu và biến đổi đặc trưng. API DataFrame cùng bộ tối ưu truy vấn *Catalyst* giúp các phép biến đổi được thực thi hiệu quả; kỹ thuật **broadcast join** được dùng để ghép tập tương tác nhỏ với bảng dữ liệu lớn mà không gây xáo trộn (shuffle) tốn kém.

2.3.2 Định dạng cột Apache Parquet

Sau tiền xử lý, dữ liệu được lưu dưới định dạng **Apache Parquet** – một định dạng lưu trữ *theo cột* (columnar). So với định dạng dòng (CSV, JSON), Parquet mang lại các ưu điểm then chốt cho dữ liệu lớn:

- **Nén hiệu quả:** dữ liệu cùng cột có kiểu giống nhau nên nén tốt, giảm đáng kể dung lượng lưu trữ.
- **Đọc chọn lọc:** chỉ cần đọc các cột liên quan, giảm I/O.
- **Tốc độ đọc/ghi cao:** phù hợp với các tác vụ phân tích và huấn luyện lặp lại.

Để tăng tốc độ nạp dữ liệu khi huấn luyện, đề án áp dụng kỹ thuật **nạp dữ liệu đa luồng bất đồng bộ** (asynchronous multi-worker loading) thông qua DataLoader, cho phép phân cứng CPU chuẩn bị batch tiếp theo trong khi GPU đang tính toán batch hiện tại.

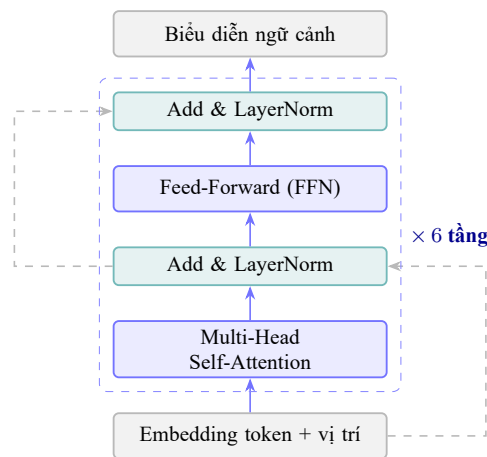
2.4 Mô hình ngôn ngữ và biểu diễn văn bản

2.4.1 Kiến trúc Transformer

Mô hình ngôn ngữ hiện đại dựa trên kiến trúc **Transformer** với cơ chế *tự chú ý* (self-attention), cho phép mô hình nắm bắt mối quan hệ ngữ nghĩa giữa các từ trong văn bản bất kể khoảng cách. Cơ chế chú ý tính trọng số liên kết giữa các token dựa trên ba ma trận truy vấn (Query), khóa (Key) và giá trị (Value):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V, \quad (2.2)$$

trong đó d_k là số chiều của vector khóa, đóng vai trò hệ số tỷ lệ giúp ổn định gradient. Mỗi tầng Transformer (Hình 2.1) gồm một khối *chú ý đa đầu* (multi-head attention) và một mạng *truyền thẳng* (feed-forward), mỗi khối kèm kết nối tắt (residual) và chuẩn hóa lớp (layer norm). MiniLM xếp chồng 6 tầng như vậy. Nhờ tính toán song song trên toàn bộ chuỗi, Transformer học được biểu diễn ngữ cảnh phong phú.



Hình 2.1: Cấu trúc một tầng mã hóa Transformer (MiniLM xếp chồng 6 tầng)

Đồ án sử dụng mô hình all-MiniLM-L6-v2 thuộc họ *Sentence-Transformers* – một mô hình gọn nhẹ (6 tầng, đầu ra 384 chiều, khoảng 22,7 triệu tham số) được chưng cất (distillation) và tinh chỉnh chuyên để sinh **vector câu** (sentence embedding) chất lượng cao, cân bằng tốt giữa độ chính xác và tốc độ – phù hợp với yêu cầu mã hóa hàng trăm nghìn văn bản.

2.4.2 Từ văn bản tới vector ngữ nghĩa

Văn bản mô tả sản phẩm (tiêu đề, tính năng, mô tả, danh mục...) được đưa qua mô hình để thu các vector ngữ cảnh cho từng token. Để có một vector *cố định* đại diện cho

toàn bộ văn bản, đồ án áp dụng phép **trung bình có mặt nạ** (mean pooling) trên các token:

$$\mathbf{v} = \frac{\sum_t m_t \mathbf{h}_t}{\sum_t m_t}, \quad (2.3)$$

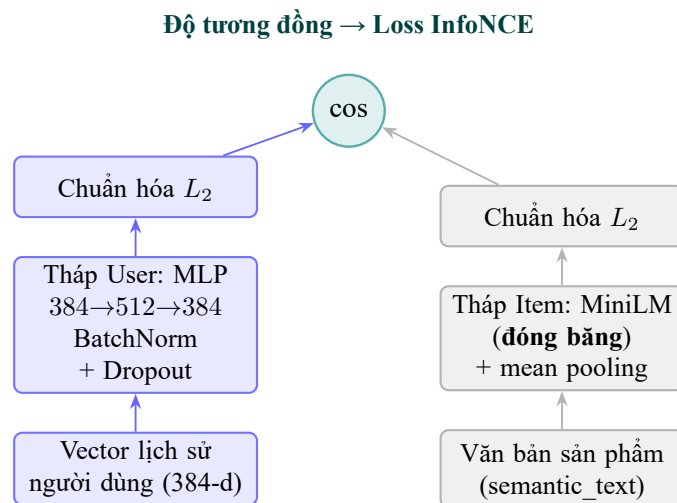
với $\mathbf{h}_t$ là vector của token thứ t và $m_t \in \{0, 1\}$ là mặt nạ chú ý (loại bỏ token đệm). Vector kết quả được chuẩn hóa L_2 . Nhờ vậy, hai sản phẩm có mô tả tương đồng về ngữ nghĩa sẽ có vector gần nhau, giải quyết được vấn đề khởi động lạnh cho sản phẩm mới.

2.4.3 Tiền huấn luyện và Sentence-BERT

Sức mạnh của bộ mã hóa đến từ quá trình **tiền huấn luyện** (pre-training) trên kho văn bản khổng lồ, giúp mô hình tích lũy tri thức ngôn ngữ tổng quát trước khi áp dụng cho bài toán cụ thể. Tuy nhiên, các vector token từ BERT gốc không tối ưu cho việc đo độ tương đồng câu. **Sentence-BERT** khắc phục bằng cách tinh chỉnh mô hình theo cặp/bộ ba câu với mục tiêu sao cho các câu cùng nghĩa có vector gần nhau – chính là tinh thần của học tương phản. Mô hình all-MiniLM-L6-v2 được huấn luyện trên hơn một tỷ cặp câu, nên vector 384 chiều mà nó sinh ra mang chất lượng ngữ nghĩa cao, rất phù hợp để làm *biểu diễn sản phẩm cố định* trong tháp Item. Đây là lý do đồ án có thể **đóng băng** tháp này mà vẫn thu được không gian biểu diễn tốt, tiết kiệm đáng kể tài nguyên huấn luyện.

2.5 Kiến trúc hai tháp (Two-Tower)

Two-Tower (hay *dual encoder*) là kiến trúc gồm hai nhánh mạng độc lập (Hình 2.2): một **tháp sản phẩm** mã hóa sản phẩm và một **tháp người dùng** mã hóa người dùng, cùng cho ra vector trong một không gian chung. Điểm mạnh của kiến trúc này là hai tháp có thể tính toán *tách rời*: toàn bộ vector sản phẩm có thể được mã hóa trước (offline) và lập chỉ mục, còn vector người dùng chỉ cần tính khi cần gợi ý – nhờ vậy hệ thống truy hồi cực nhanh trên kho sản phẩm lớn.



Hình 2.2: Kiến trúc hai tháp (Two-Tower) với tháp sản phẩm đóng băng

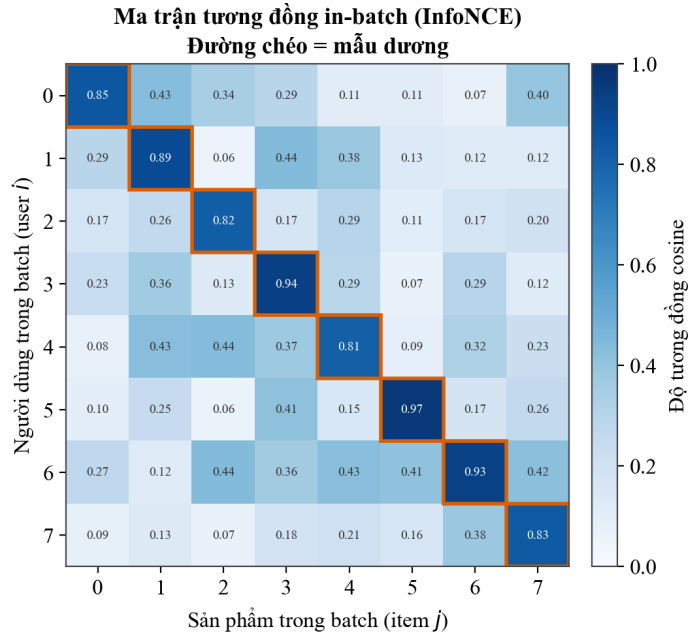
Trong thiết kế của đồ án, **tháp sản phẩm được đóng băng hoàn toàn** (không cập nhật trọng số) nhằm tiết kiệm bộ nhớ và tận dụng tri thức ngôn ngữ sẵn có; chỉ **tháp người dùng** (một mạng nơ-ron đa tầng) được huấn luyện để học cách ánh xạ lịch sử hành vi sang không gian sản phẩm.

2.6 Học tương phản và hàm mất mát InfoNCE

Học tương phản (Contrastive Learning) huấn luyện mô hình bằng cách so sánh: với mỗi người dùng, sản phẩm thực sự tương tác (mẫu dương) cần được kéo lại gần, trong khi các sản phẩm khác (mẫu âm) bị đẩy ra xa. Đồ án sử dụng hàm mất mát **InfoNCE** với chiến lược *mẫu âm trong batch* (in-batch negatives): trong một batch gồm B cặp (người dùng, sản phẩm), sản phẩm của các cặp còn lại đóng vai trò mẫu âm cho người dùng đang xét. Hàm mất mát cho người dùng i là:

$$\mathcal{L}_i = -\log \frac{\exp(\cos(\mathbf{u}_i, \mathbf{v}_i)/\tau)}{\sum_{j=1}^B \exp(\cos(\mathbf{u}_i, \mathbf{v}_j)/\tau)}, \quad (2.4)$$

trong đó $\mathbf{v}_i$ là sản phẩm dương của người dùng i , $\mathbf{v}_j$ là các sản phẩm trong batch, và τ là *tham số nhiệt độ* (temperature) điều chỉnh độ "sắc" của phân phối. Về bản chất, đây là một bài toán phân loại B lớp mà nhãn đúng nằm trên đường chéo của ma trận tương đồng – cài đặt hiệu quả bằng CrossEntropyLoss. Chiến lược in-batch negatives giúp tận dụng tối đa mỗi batch mà không cần lấy mẫu âm tường minh, rất phù hợp với huấn luyện quy mô lớn. Hình 2.3 minh họa trực quan: ma trận tương đồng $B \times B$ giữa mọi người dùng và mọi sản phẩm trong batch, trong đó nhiệm vụ huấn luyện là làm cho các ô *đường chéo* (cặp đúng) có giá trị cao nhất trên mỗi hàng.



Hình 2.3: Ma trận tương đồng in-batch của InfoNCE; đường chéo là cặp dương cần tối đa hóa

2.6.1 Vai trò của tham số nhiệt độ

Tham số nhiệt độ τ kiểm soát mức độ "gay gắt" của hàm mất mát. Khi τ nhỏ, mô hình *trừng phạt mạnh* những mẫu âm khó (nằm gần mẫu dương), thúc đẩy phân tách rạch ròi trong không gian biểu diễn; khi τ lớn, gradient trải đều và mô hình "khoan dung" hơn. Về mặt lý thuyết, học tương phản tối ưu đồng thời hai tính chất: **alignment** (các cặp dương phải gần nhau) và **uniformity** (các biểu diễn phải trải đều trên mặt cầu đơn vị để giữ nhiều thông tin). Giá trị $\tau = 0,1$ được đề án lựa chọn là một mức cân bằng phổ biến và hiệu quả cho bài toán gợi ý.

2.7 Tìm kiếm láng giềng gần nhất

Sau khi huấn luyện, việc gợi ý Top- N quy về tìm các sản phẩm có vector gần vector người dùng nhất. Với kho chỉ vài chục nghìn sản phẩm, ta có thể tính tương đồng *vét cạn* (brute-force) như trong giao thức đánh giá của đề án. Tuy nhiên, khi kho lên tới hàng triệu sản phẩm, phép vét cạn trở nên quá tốn kém cho yêu cầu thời gian thực. Khi đó, các kỹ thuật **tìm kiếm láng giềng gần nhất xấp xỉ** (Approximate Nearest Neighbor – ANN) như chỉ mục đảo (IVF), lượng tử hóa tích (product quantization) hay đồ thị HNSW – được hiện thực trong thư viện **FAISS** – cho phép truy hồi trong vài phần nghìn giây với độ chính xác gần như không suy giảm. Kiến trúc hai tháp đặc biệt phù hợp với ANN vì toàn bộ vector sản phẩm có thể được lập chỉ mục trước. Đây là nền tảng cho hướng phát

triển triển khai thời gian thực ở Chương 5.

2.8 Các độ đo đánh giá

Vì bài toán là truy hồi Top- N , đồ án dùng hai độ đo xếp hạng phổ biến, tính trên **toàn bộ kho sản phẩm**. Với mỗi người dùng, gọi Rel_u là tập sản phẩm thực sự liên quan và Rec_u^K là Top- K sản phẩm được gợi ý:

$$\text{Recall@K} = \frac{|\text{Rec}_u^K \cap \text{Rel}_u|}{|\text{Rel}_u|}. \quad (2.5)$$

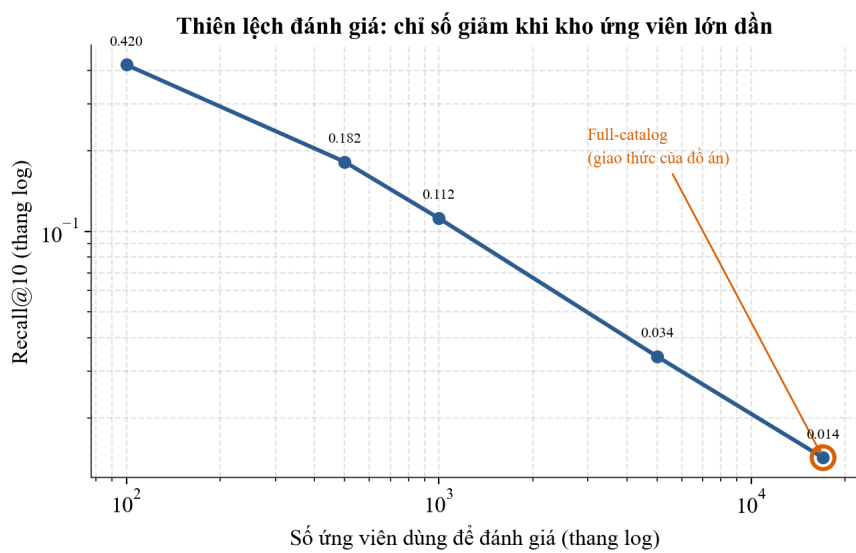
Trong giao thức leave-one-out (mỗi người dùng có đúng một sản phẩm mục tiêu), Recall@K rút gọn thành tỷ lệ người dùng có sản phẩm mục tiêu nằm trong Top- K . **NDCG@K** bổ sung yếu tố thứ hạng, thưởng nhiều hơn khi sản phẩm đúng xuất hiện ở vị trí cao:

$$\text{NDCG@K} = \frac{1}{\log_2(p+1)}, \quad (2.6)$$

với p là vị trí của sản phẩm mục tiêu trong danh sách gợi ý (nếu nằm trong Top- K , ngược lại bằng 0). Việc đánh giá trên toàn kho ~ 17.000 sản phẩm khiến các con số tuyệt đối nhỏ, nhưng phản ánh *trung thực* độ khó thực tế và cho phép so sánh công bằng với baseline chọn ngẫu nhiên.

2.8.1 Toàn kho và thiên lệch lấy mẫu

Nhiều nghiên cứu đánh giá theo cách *lấy mẫu* (sampled metric): trộn sản phẩm mục tiêu với một số ít mẫu âm (ví dụ 99 hoặc 999 sản phẩm) rồi xếp hạng trong nhóm nhỏ đó. Cách này nhanh nhưng gây **thiên lệch lạc quan**: kho ứng viên càng nhỏ, chỉ số càng cao một cách giả tạo. Hình 2.4 minh họa hiện tượng này – cùng một mô hình, Recall@10 giảm mạnh từ khoảng 0,42 (chỉ 100 ứng viên) xuống 0,0142 khi xếp hạng trên toàn bộ 17.000 sản phẩm. Do đó, đồ án chủ trương đánh giá **toàn kho (full-catalog)** để có con số trung thực, dù vất vả hơn.



Hình 2.4: Recall@10 giảm khi số ứng viên đánh giá tăng – minh chứng thiên lệch lấy mẫu

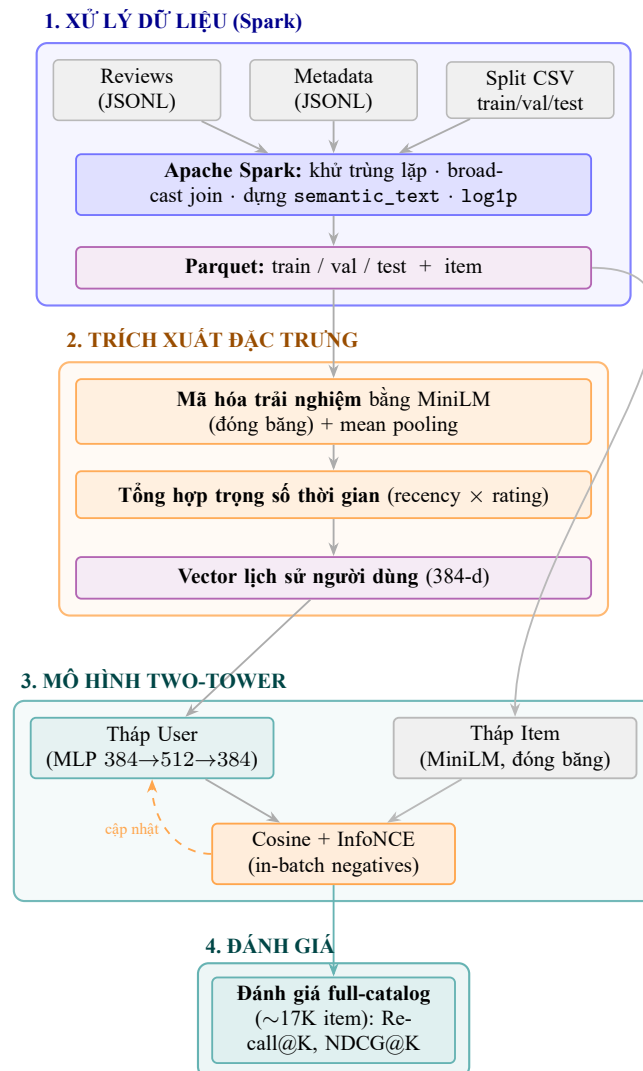
Chương 3

Phân tích và thiết kế hệ thống

3.1 Kiến trúc tổng thể

Hệ thống được thiết kế thành một pipeline gồm bốn giai đoạn nối tiếp, từ dữ liệu thô tới danh sách gợi ý (Hình 3.1): (1) tiền xử lý dữ liệu lớn bằng Spark và lưu Parquet; (2) mã hóa ”trải nghiệm” người dùng và tổng hợp lịch sử có trọng số thời gian; (3) huấn luyện mô hình hai tháp với học tương phản; (4) đánh giá truy hồi trên toàn bộ kho sản phẩm.

Hình 3.1 là sơ đồ khối chi tiết toàn bộ luồng – từ dữ liệu thô, qua xử lý dữ liệu lớn, trích xuất đặc trưng, tới mô hình và đánh giá – kèm các tệp/đối tượng trung gian (Parquet, vector lịch sử) chuyển giao giữa các giai đoạn.



Hình 3.1: Sơ đồ khối end-to-end: từ xử lý dữ liệu đến mô hình và đánh giá

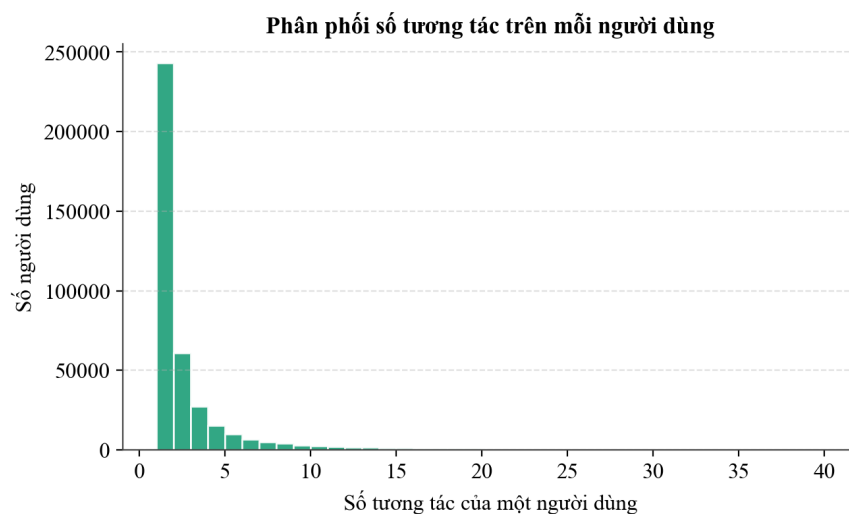
3.2 Dữ liệu và tiền xử lý bằng PySpark

Dữ liệu sử dụng là **Amazon Reviews 2023** (nhóm McAuley Lab, UCSD), danh mục *Software*, gồm hai phần: tập **tương tác** (review của người dùng, các trường `user_id`, `parent_asin`, `rating`, `title`, `text`, `timestamp`) và tập **metadata sản phẩm** (tiêu đề, tính năng, mô tả, danh mục, giá, điểm trung bình...). Quy mô bài toán được tổng hợp trong Bảng 3.1.

Bảng 3.1: Thống kê quy mô bộ dữ liệu Amazon Software

Thuộc tính	Giá trị
Số người dùng	146.396
Số sản phẩm phần mềm độc nhất	> 17.000
Số tương tác huấn luyện (train)	≈ 984.000
Số mẫu kiểm định (validation)	146.396 (1 / người dùng)
Số mẫu kiểm thử (test)	146.396 (1 / người dùng)
Số chiều vector biểu diễn	384
Thang điểm đánh giá	1 – 5 sao

Theo giao thức leave-last-out, mỗi người dùng đóng góp đúng một mẫu cho tập kiểm định và một mẫu cho tập kiểm thử, phần còn lại thuộc tập huấn luyện. Phân phối số tương tác trên mỗi người dùng (Hình 3.2) lệch phải mạnh – đa số người dùng chỉ có vài tương tác, một thiểu số rất tích cực – phản ánh tính thưa thớt và đuôi dài đặc trưng của dữ liệu thương mại điện tử.



Hình 3.2: Phân phối số tương tác trên mỗi người dùng

3.2.1 Làm sạch và kiểm tra chất lượng dữ liệu

Dữ liệu thô từ Amazon chứa nhiều giá trị thiếu và bản ghi trùng. Quy trình làm sạch trên Spark gồm: (i) **khai báo lược đồ trường minh** (StructType) khi đọc metadata để kiểm soát kiểu dữ liệu của các trường phức tạp (mảng, map); (ii) **đếm giá trị thiếu** từng cột để đánh giá chất lượng; (iii) **khử trùng lặp** bằng dropDuplicates; và (iv) **ghép nối** tập chia chuẩn với dữ liệu đầy đủ bằng broadcast join – kỹ thuật phát tán bảng nhỏ tới mọi executor, tránh thao tác shuffle tốn kém trên bảng lớn.

Listing 3.1: Kiểm tra null và ghép nối broadcast trên Spark

```
1 from pyspark.sql.functions import col, count, when, isnan, broadcast
2
3 # Dem so gia tri thieu tren tung cot
4 df.select([
5     count(when(col(c).isNull(), c)).alias(c) for c in df.columns
6 ]).show()
7
8 # Khu trung lap
9 df = df.dropDuplicates()
10
11 # Ghep tap split chuan (nho) voi du lieu day du (lon) bang broadcast join
12 df_train_ok = broadcast(df_train).join(
13     df, on=["user_id", "parent_asin", "timestamp"], how="left")
```

Các trường số như price và rating_number có phân phối lệch phải rất mạnh nên được biến đổi log1p để nén khoảng giá trị, giúp ổn định khi đưa vào mô hình. Cuối cùng, tập tương tác được lọc để chỉ giữ lại các sản phẩm tồn tại trong bảng metadata, bảo đảm mọi sản phẩm đều có văn bản ngữ nghĩa.

3.2.2 Dựng văn bản ngữ nghĩa cho sản phẩm

Bước quan trọng nhất của tiền xử lý là tổng hợp các trường metadata rồi rạc thành một chuỗi văn bản ngữ nghĩa thống nhất (semantic_text) để làm đầu vào cho mô hình ngôn ngữ. Các trường dạng mảng được ghép bằng concat_ws, trường details dạng map được duyệt thành các dòng ”khóa: giá trị”:

Listing 3.2: Dựng semantic_text từ metadata bằng PySpark

```
1 from pyspark.sql import functions as F
2
3 # Ghep cac truong mang (features, description, categories) thanh text
4 for c in ["features", "description", "categories"]:
5     df_i = df_i.withColumn(f"{c}_text",
6         F.when(F.col(c).isNotNull(), F.concat_ws(" ",
7             ↪ F.col(c))).otherwise(F.lit("")))
7
```

```

8 # Tổng hợp semantic_text có cấu trúc theo từng khối
9 df_i = df_i.withColumn("semantic_text", F.concat_ws("\n\n",
10     F.when(F.col("title") != "", F.concat(F.lit("TITLE\n"),
11         ⇨ F.col("title"))),
12     F.when(F.col("features_text") != "", F.concat(F.lit("FEATURES\n"),
13         ⇨ F.col("features_text"))),
14     F.when(F.col("description_text") != "", F.concat(F.lit("DESCRIPTION\n"),
15         ⇨ F.col("description_text"))),
16     F.when(F.col("categories_text") != "", F.concat(F.lit("CATEGORIES\n"),
17         ⇨ F.col("categories_text")))))
18
19 # Chuẩn hóa trường số: log1p cho giá và số lượng đánh giá (giảm lệch phải)
20 df_i = df_i.withColumn("price", F.log1p(F.col("price")))
21 df_i = df_i.withColumn("rating_number", F.log1p(F.col("rating_number")))

```

Một ví dụ semantic_text sau khi tổng hợp có dạng như sau:

Listing 3.3: Ví dụ văn bản ngữ nghĩa của một sản phẩm phần mềm

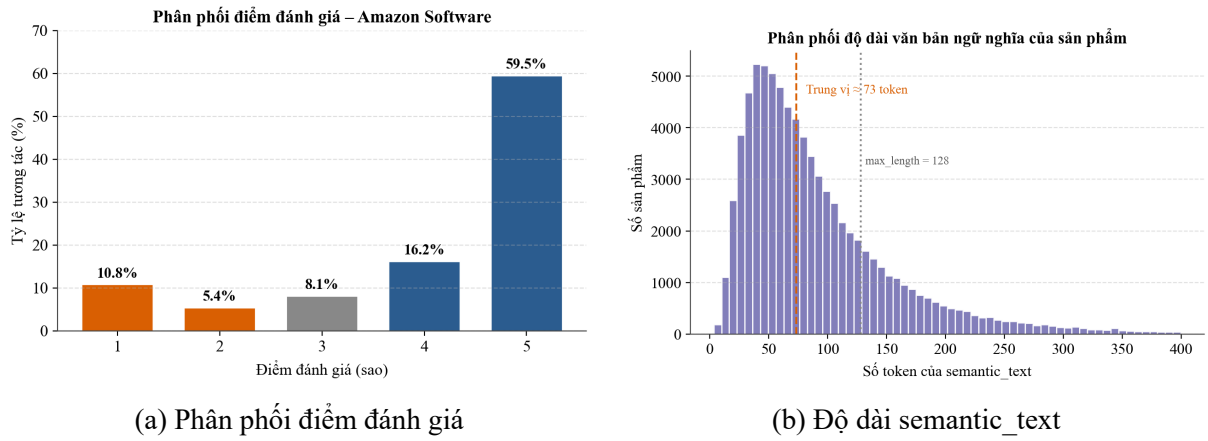
```

1 TITLE
2 Norton 360 Deluxe 2024, Antivirus software for 5 Devices
3
4 FEATURES
5 Real-time protection against malware, ransomware and phishing
6 Secure VPN for online privacy; 50 GB cloud backup
7
8 CATEGORIES
9 Software | Antivirus & Security | Internet Security Suites

```

Khi đưa vào mô hình ngôn ngữ, toàn bộ khối văn bản này được nén thành một vector 384 chiều duy nhất, ”hiểu” được rằng đây là phần mềm bảo mật – nhờ đó các sản phẩm cùng loại sẽ nằm gần nhau trong không gian biểu diễn.

Dữ liệu sau xử lý được lưu thành các tệp Parquet riêng cho tập huấn luyện, kiểm định, kiểm thử và bảng sản phẩm. Phân phối điểm đánh giá (Hình 3.3a) lệch mạnh về phía điểm cao – đặc trưng điển hình của dữ liệu thương mại điện tử. Độ dài semantic_text (Hình 3.3b) có trung vị khoảng 70–80 token; mô hình giới hạn max_length = 128 để cân bằng giữa thông tin và tốc độ.



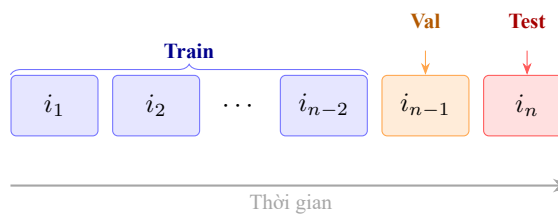
Hình 3.3: Một số đặc trưng của dữ liệu Amazon Software

3.3 Chiến lược chia dữ liệu theo thời gian

Để mô phỏng sát kịch bản thực tế (dự đoán hành vi *tương lai* từ *quá khứ*), đồ án áp dụng phương pháp **cắt chuỗi hành vi theo thời gian** (leave-last-out) cho mỗi người dùng:

- **Tập kiểm thử (test):** sản phẩm được tương tác *cuối cùng*.
- **Tập kiểm định (validation):** sản phẩm *kề cuối*.
- **Tập huấn luyện (train):** toàn bộ các tương tác *trước đó*.

Cách chia này nghiêm ngặt hơn chia ngẫu nhiên vì tuyệt đối không để lộ thông tin tương lai vào quá khứ (Hình 3.4). Lịch sử của tập kiểm định kế thừa từ tập huấn luyện; lịch sử của tập kiểm thử được tính trên hợp của tập huấn luyện và kiểm định.



Hình 3.4: Chia dữ liệu leave-last-out: mục cuối làm test, kề cuối làm val, còn lại train

3.4 Mã hóa trải nghiệm và tổng hợp lịch sử người dùng

3.4.1 Mã hóa trải nghiệm

Trước hết, mỗi sản phẩm được biểu diễn bằng một chuỗi văn bản tổng hợp (item_text_map) ghép semantic_text với các thuộc tính bổ sung (danh mục,

cửa hàng, giá, điểm và số lượng đánh giá):

Listing 3.4: Dựng từ điển văn bản sản phẩm item_text_map

```
1 item_text_map = {}
2 for r in item_df.itertuples():
3     item_text_map[r.parent_asin] = (
4         f"{safe(r.semantic_text)}\n"
5         f"Category: {safe(r.main_category)}\nStore: {safe(r.store)}\n"
6         f"Price: {safe(r.price)}\nAverage Rating: {safe(r.average_rating)}\n"
7         f"Rating Count: {safe(r.rating_number)}")
```

Mỗi tương tác của người dùng được biến thành một "trải nghiệm" bằng cách ghép văn bản sản phẩm trên với tiêu đề và nội dung review, rồi mã hóa qua mô hình MiniLM (đóng băng) theo từng batch trên GPU:

Listing 3.5: Mã hóa trải nghiệm người dùng hàng loạt trên GPU

```
1 def precompute_user_experience_vectors(df, batch_size=256):
2     base = df['parent_asin'].map(item_text_map).fillna('None')
3     combined = (base + "\nUser Review Title: " + df['title'].fillna('None')
4                 + "\nUser Review Content: " +
5                 df['text'].fillna('None')).tolist()
6     vectors = []
7     with torch.no_grad():
8         for i in range(0, len(combined), batch_size):
9             enc = tokenizer(combined[i:i+batch_size], padding=True, truncation=True,
10                             max_length=128, return_tensors="pt").to(device)
11             out = base_model(**enc)
12             emb = F.normalize(mean_pooling(out, enc["attention_mask"]), dim=1)
13             vectors.append(emb.cpu().numpy())
14     return np.vstack(vectors)
```

3.4.2 Tổng hợp lịch sử với trọng số thời gian

Vector đại diện cho một người dùng là *trung bình có trọng số* của các vector trải nghiệm trong lịch sử. Trọng số kết hợp hai yếu tố: (i) **độ gần đây** (recency) – tương tác càng mới càng quan trọng, theo hàm suy giảm mũ $e^{-(n-i)/n}$; và (ii) **mức độ ưa thích** – điểm rating càng cao trọng số càng lớn, theo hệ số $(1 + \text{rating}/5)$:

$$\mathbf{u} = \frac{\sum_{i=1}^n w_i \mathbf{e}_i}{\sum_{i=1}^n w_i}, \quad w_i = e^{-(n-i)/n} \cdot \left(1 + \frac{\text{rating}_i}{5}\right), \quad (3.1)$$

trong đó $\mathbf{e}_i$ là vector trải nghiệm thứ i và n là số tương tác của người dùng. Vector kết quả được chuẩn hóa L_2 . Hình 3.5 minh họa cách trọng số tăng dần về phía các tương tác gần đây nhất. Toàn bộ phép tổng hợp này chạy trên CPU/RAM (gom nhóm theo user_id), độc lập với GPU, nên rất nhẹ và nhanh.

Listing 3.6: Tổng hợp vector lịch sử có trọng số thời gian

```

1 def aggregate_user_history(df, experience_vectors):
2     df['_vec'] = list(experience_vectors)
3     user_hist = {}
4     for uid, g in df.groupby("user_id"):
5         g = g.reset_index(drop=True); n = len(g)
6         vectors = np.array(g['_vec'].tolist())
7         idx = np.arange(n)
8         recency = np.exp(-(n - idx) / max(n, 1))           # càng mới càng nặng
9         ratings = g['rating'].fillna(0).to_numpy()
10        weights = (recency * (1 + ratings / 5.0)).reshape(-1, 1)
11        u = np.sum(vectors * weights, axis=0) / np.sum(weights)
12        user_hist[uid] = u / (np.linalg.norm(u) + 1e-9)    # chuẩn hóa L2
13    return user_hist

```

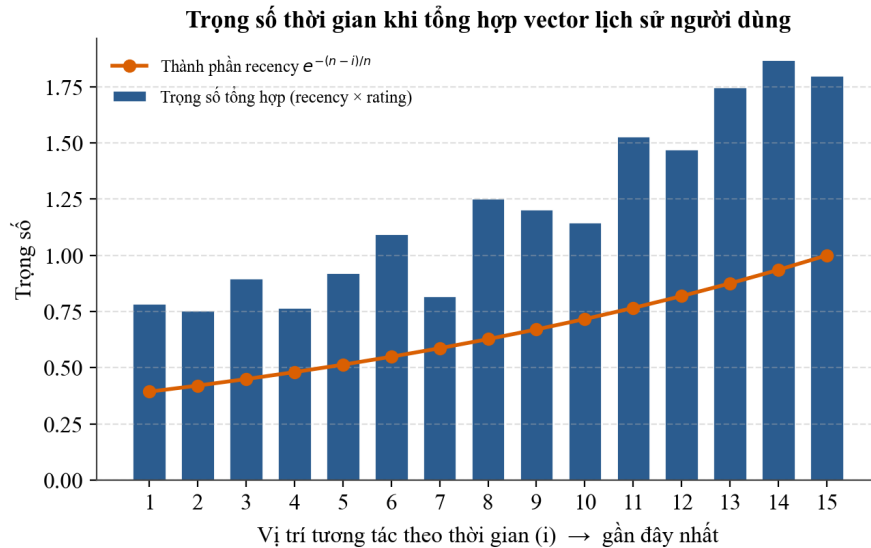
Phép **mean pooling có mặt nạ** (dùng ở cả khâu mã hóa) lấy trung bình các vector token, bỏ qua token đệm nhờ attention_mask:

Listing 3.7: Mean pooling có mặt nạ

```

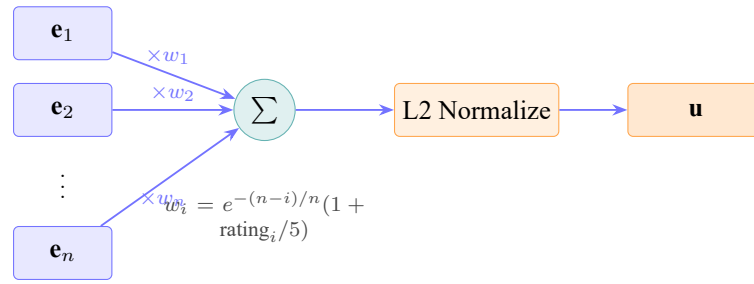
1 def mean_pooling(out, mask):
2     token = out.last_hidden_state                       # [B, L, 384]
3     mask = mask.unsqueeze(-1).expand(token.size()).float()
4     return (token * mask).sum(1) / torch.clamp(mask.sum(1), min=1e-9)

```



Hình 3.5: Trọng số thời gian khi tổng hợp vector lịch sử người dùng ($n = 15$)

Toàn bộ luồng tổng hợp được tóm tắt trong Hình 3.6: các vector trải nghiệm được nhân với trọng số tương ứng, cộng lại rồi chuẩn hóa thành một vector người dùng duy nhất.



Hình 3.6: Luồng tổng hợp vector lịch sử người dùng từ các vector trải nghiệm

3.5 Thiết kế mô hình hai tháp

Mô hình TwoTowerVectorModel hiện thực hóa kiến trúc ở Mục 2.5. **Tháp người dùng** là một mạng MLP biến đổi vector lịch sử 384 chiều, có BatchNorm để ổn định gradient và Dropout để chống quá khớp. **Tháp sản phẩm** là MiniLM đóng băng, chỉ làm nhiệm vụ mã hóa văn bản.

Listing 3.8: Mô hình hai tháp (tháp sản phẩm đóng băng)

```

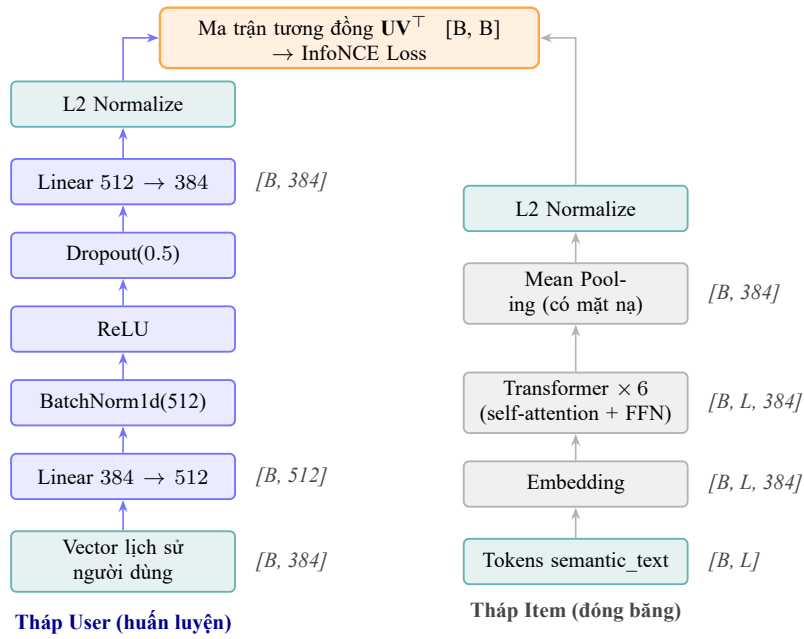
1 class TwoTowerVectorModel(nn.Module):
2     def __init__(self, model_name):
3         super().__init__()
4         # Tháp User: MLP có thể huấn luyện
5         self.user_projection = nn.Sequential(
6             nn.Linear(384, 512), nn.BatchNorm1d(512), nn.ReLU(),
7             nn.Dropout(0.5), nn.Linear(512, 384))
8         # Tháp Item: đóng băng hoàn toàn
9         self.item_encoder = AutoModel.from_pretrained(model_name)
10        for p in self.item_encoder.parameters():
11            p.requires_grad = False
12
13        def forward_user(self, user_vectors):
14            return F.normalize(self.user_projection(user_vectors), dim=1)
15
16        def forward_item(self, tokens):
17            with torch.no_grad():
18                out = self.item_encoder(**tokens)
19                emb = self.mean_pooling(out, tokens["attention_mask"])
20            return F.normalize(emb, dim=1)

```

3.5.1 Sơ đồ kiến trúc chi tiết

Hình 3.7 mô tả chi tiết từng tầng của mô hình cùng kích thước tensor tương ứng (B là kích thước batch, L là độ dài chuỗi token). Tháp người dùng biến đổi vector lịch sử qua một khối MLP, còn tháp sản phẩm đưa chuỗi token qua 6 tầng Transformer của MiniLM rồi gộp trung bình. Hai vector đầu ra cùng kích thước 384 chiều được chuẩn hóa và đem

so khớp bằng tích vô hướng.



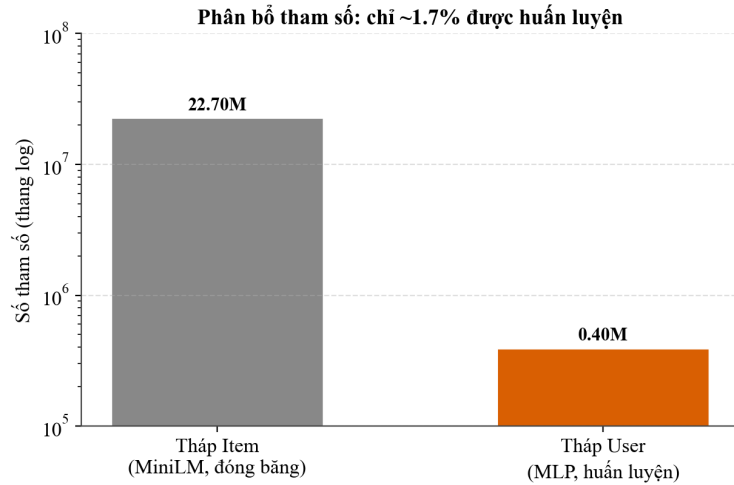
Hình 3.7: Sơ đồ kiến trúc chi tiết của mô hình hai tháp kèm kích thước tensor

3.5.2 Phân tích số tham số

Một ưu điểm nổi bật của thiết kế là **hiệu quả tham số**. Bảng 3.2 liệt kê số tham số của tháp người dùng – chỉ khoảng 395.000 tham số khả huấn luyện, trong khi tháp sản phẩm (MiniLM) có $\sim 22,7$ triệu tham số nhưng được **đóng băng hoàn toàn**. Như vậy, chỉ khoảng 1,7% tổng số tham số thực sự được cập nhật (Hình 3.8), giúp giảm mạnh chi phí bộ nhớ và thời gian huấn luyện.

Bảng 3.2: Số tham số khả huấn luyện của tháp người dùng

Tầng	Số tham số
Linear 384 → 512	197.120
BatchNorm1d(512)	1.024
Linear 512 → 384	196.992
Tổng (khả huấn luyện)	395.136
Tháp Item (MiniLM, đóng băng)	$\approx 22.700.000$



Hình 3.8: Phân bố tham số giữa hai tháp (chỉ ~ 1,7% được huấn luyện)

3.6 Hàm mất mát và cấu hình huấn luyện

Mô hình được huấn luyện bằng hàm mất mát **InfoNCE** (Mục 2.6) với mẫu âm trong batch. Ma trận tương đồng giữa mọi người dùng và mọi sản phẩm trong batch được tính, nhân đúng nằm trên đường chéo:

Listing 3.9: Hàm mất mát InfoNCE với mẫu âm trong batch

```

1 class InfoNCE(nn.Module):
2     def __init__(self, temperature=0.1):
3         super().__init__()
4         self.temperature = temperature
5         self.ce = nn.CrossEntropyLoss()
6
7     def forward(self, user_out, item_out):
8         sim = torch.matmul(user_out, item_out.T) / self.temperature
9         labels = torch.arange(user_out.size(0), device=user_out.device)
10        return self.ce(sim, labels)
11
12 loss_fn = InfoNCE(temperature=0.1)
13 optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=5e-2)

```

Mô hình được tối ưu bằng AdamW (learning rate 10^{-4} , weight decay 0,05). Quá trình huấn luyện áp dụng **tính toán độ chính xác hỗn hợp** (mixed precision) để tiết kiệm bộ nhớ GPU và tăng tốc, kèm cơ chế dừng sớm (early stopping) dựa trên mất mát kiểm định. Chi tiết cấu hình và kết quả được trình bày ở Chương 4.

Chương 4

Kết quả thực nghiệm

4.1 Môi trường và cấu hình

Toàn bộ thực nghiệm được triển khai trên môi trường điện toán đám mây **Google Colab** với bộ xử lý đồ họa (GPU) gia tốc. Để tối ưu vùng nhớ và tốc độ, đồ án sử dụng kỹ thuật **tính toán độ chính xác hỗn hợp** (mixed precision) thông qua `torch.amp` kết hợp `GradScaler`. Bảng 4.1 tổng hợp các siêu tham số chính.

Bảng 4.1: Cấu hình môi trường và siêu tham số huấn luyện

Thành phần	Cấu hình
Bộ mã hóa văn bản (tháp Item)	all-MiniLM-L6-v2 (6 tầng, 384 chiều), đóng băng
Tháp người dùng	MLP 384→512→384, BatchNorm, ReLU, Dropout 0,5
Hàm mất mát	InfoNCE (in-batch negatives), temperature $\tau = 0,1$
Bộ tối ưu	AdamW, learning rate 10^{-4} , weight decay 0,05
Batch size	256
Số epoch	5 (early stopping theo val loss)
Độ chính xác	Mixed precision (FP16) trên GPU
Nạp dữ liệu	Parquet + DataLoader 2 worker, bất đồng bộ

Quy mô dữ liệu: khoảng 984.000 tương tác huấn luyện (3.844 batch mỗi epoch), 146.396 người dùng và hơn 17.000 sản phẩm. Bảng 4.2 phân rã thời gian một epoch: phần lớn dành cho huấn luyện (~ 38 phút), phần còn lại cho tính mất mát kiểm định và đánh giá toàn kho (~ 11 phút).

Bảng 4.2: Phân rã thời gian một epoch (tốc độ $\sim 1,7$ batch/giây)

Giai đoạn	Thời gian
Huấn luyện (3.844 batch)	≈ 38 phút
Tính mất mát kiểm định (572 batch)	$\approx 5,7$ phút
Mã hóa kho item + tính Recall/NDCG	$\approx 5,4$ phút
Tổng mỗi epoch	≈ 49 phút

4.2 Chi tiết vòng lặp huấn luyện

Vòng lặp huấn luyện kết hợp nhiều kỹ thuật để tối ưu tốc độ và bộ nhớ. Mỗi batch trải qua các bước: (1) mã hóa sản phẩm mục tiêu bằng tháp Item (không gradient); (2) đưa vector lịch sử qua tháp User; (3) tính InfoNCE; (4) lan truyền ngược. Toàn bộ phép tính tiến (forward) được bọc trong autocast để dùng FP16, còn GradScaler nhân tỷ lệ mất mát nhằm tránh tràn số (underflow) khi gradient quá nhỏ ở độ chính xác thấp.

Listing 4.1: Vòng lặp huấn luyện với mixed precision và early stopping

```

1 scaler = torch.amp.GradScaler('cuda')
2 best_val_loss = float('inf'); patience_counter = 0
3
4 for epoch in range(EPOCHS):
5     model.train(); total_loss = 0
6     for batch in train_loader:
7         item_tokens = {k: v.to(device) for k, v in batch["item_tokens"].items()}
8         optimizer.zero_grad()
9         with torch.amp.autocast('cuda'):           # mixed precision FP16
10             item_out = model.forward_item(item_tokens)           # tháp dòng bang
11             user_out = model.forward_user(batch["user_vectors"].to(device))
12             loss = loss_fn(user_out, item_out)                   # InfoNCE
13             scaler.scale(loss).backward()                       # scale tránh underflow
14             scaler.step(optimizer); scaler.update()
15             total_loss += loss.item()
16
17 val_loss = compute_loss(model, val_loader, loss_fn, device)
18 if val_loss < best_val_loss:                                   # early stopping theo val loss
19     best_val_loss = val_loss; patience_counter = 0
20     torch.save(model.state_dict(), best_model_path)
21 else:
22     patience_counter += 1
23     if patience_counter >= PATIENCE: break

```

Cơ chế **early stopping** lưu lại trọng số tốt nhất mỗi khi mất mát kiểm định giảm và dừng huấn luyện nếu không cải thiện sau PATIENCE epoch – bảo đảm mô hình cuối cùng

là phiên bản tổng quát hóa tốt nhất, tránh lãng phí tính toán.

4.3 Thiết kế nạp dữ liệu theo batch

Việc nạp dữ liệu được tổ chức qua lớp Dataset trả về vector người dùng (đã tính sẵn) cùng văn bản sản phẩm mục tiêu, và một hàm `collate` gom batch: chỉ *văn bản sản phẩm* mới cần tokenize tại thời điểm tạo batch, còn vector người dùng đã được mã hóa trước nên chỉ việc xếp chồng – giảm đáng kể khối lượng tính toán lặp lại. DataLoader dùng 2 worker và `pin_memory` để nạp bất đồng bộ, che giấu độ trễ I/O sau thời gian tính toán của GPU.

Listing 4.2: Hàm collate chỉ tokenize nhánh sản phẩm

```
1 def collate_vector_fn(batch):
2     user_vectors = torch.stack([b["user_vector"] for b in batch]) # [B, 384]
3     item_texts = [b["item_text"] for b in batch]
4     item_tokens = tokenizer(item_texts, padding=True, truncation=True,
5                             max_length=128, return_tensors="pt")
6     return {"user_vectors": user_vectors, "item_tokens": item_tokens,
7           "target": [b["target"] for b in batch]}
```

4.4 Giao thức đánh giá toàn kho (Full-catalog)

Một đóng góp quan trọng về mặt phương pháp là cách đánh giá. Nhiều nghiên cứu chỉ đánh giá trên một nhóm nhỏ mẫu âm lấy ngẫu nhiên (in-batch / sampled metrics), dẫn tới chỉ số bị *thối phồng* và không phản ánh độ khó thực tế. Đồ án triệt tiêu hoàn toàn thiên lệch này bằng giao thức **full-catalog**:

1. Mã hóa trước **toàn bộ** hơn 17.000 sản phẩm thành ma trận vector sản phẩm.
2. Với mỗi người dùng, tính độ tương đồng với *tất cả* sản phẩm trong kho, lấy Top-50.
3. Đo Recall@K và NDCG@K bằng cách kiểm tra sản phẩm mục tiêu thật có nằm trong Top-K hay không.

Việc tính toán được chia theo cụm người dùng (1.000 người mỗi lần) để tránh tràn bộ nhớ GPU. Đây là cách đánh giá *khắt khe và trung thực* nhất, vì người dùng phải cạnh tranh với toàn bộ kho sản phẩm.

Về mặt kỹ thuật, nếu nhân trực tiếp ma trận người dùng (146.396×384) với ma trận sản phẩm (17.000×384) sẽ tạo ra ma trận điểm số khổng lồ vượt quá VRAM. Giải pháp là **xử lý theo cụm**: kho sản phẩm được mã hóa trước thành một ma trận cố định; sau đó

với mỗi cụm 1.000 người dùng, ta tính điểm số, lấy topk rồi giải phóng bộ nhớ trước khi sang cụm tiếp theo.

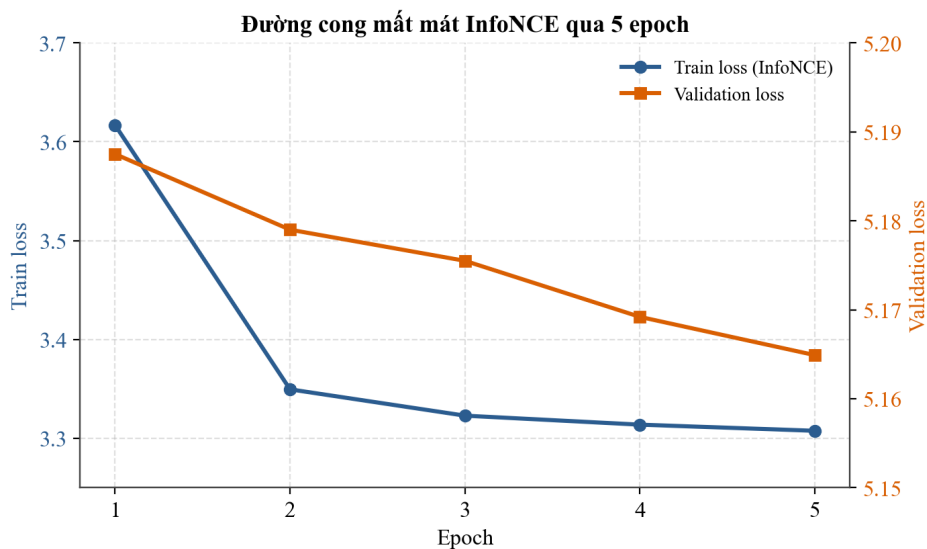
Listing 4.3: Đánh giá toàn kho theo cụm để tránh tràn VRAM

```
1 # Ma hoa truoc TOAN BO kho item mot lan
2 all_item_emb = encode_all_items(item_ids, model)          # [N_items, 384]
3
4 for i in range(0, num_users, 1000):                      # xu ly 1000 user/lan
5     user_chunk = all_user_emb[i:i+1000].to(device)
6     scores = torch.matmul(user_chunk, all_item_emb.T)     # [1000, N_items]
7     _, topk = torch.topk(scores, k=50, dim=1)             # lay Top-50
8     hits = (topk == target_chunk.view(-1, 1))            # vi tri trung
9     recall10 += hits[:, :10].any(dim=1).float().sum().item()
10    # ... cong don NDCG theo thu hang ...
11    del scores, topk; torch.cuda.empty_cache()            # giai phong VRAM
```

Nhờ chiến lược này, hệ thống có thể đánh giá trên toàn bộ 146.396 người dùng $\times$ 17.000 sản phẩm mà vẫn nằm trong giới hạn bộ nhớ của một GPU phổ thông.

4.5 Kết quả huấn luyện

Hình 4.1 thể hiện đường cong mất mát qua 5 epoch. Mất mát huấn luyện (InfoNCE) giảm rõ từ 3,62 xuống 3,31, trong khi mất mát kiểm định *giảm đều đặn* từ 5,19 xuống 5,16 – cho thấy mô hình hội tụ ổn định và **không bị quá khớp**. Cơ chế early stopping lưu lại mô hình tốt nhất tại mỗi lần val loss giảm; mô hình đạt trạng thái ổn định tại epoch thứ 5.



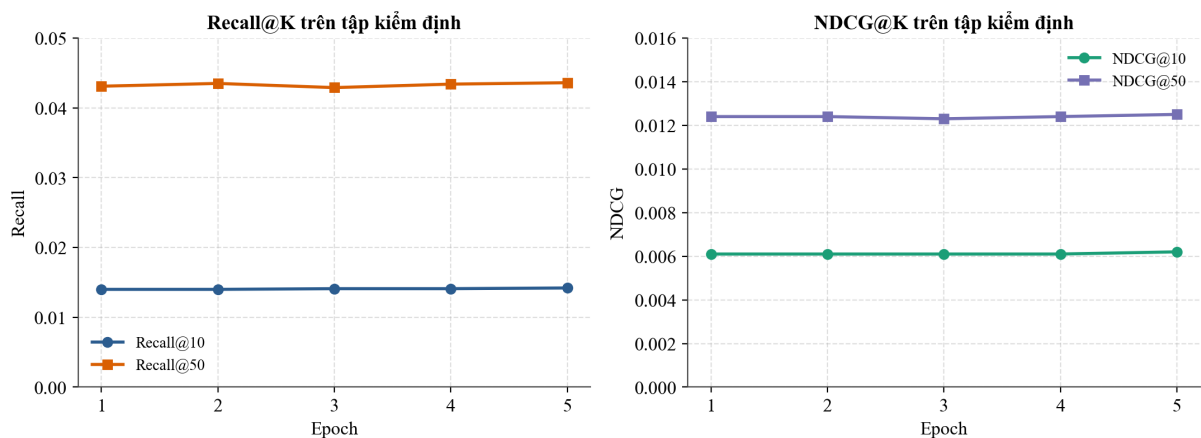
Hình 4.1: Đường cong mất mát InfoNCE trên tập huấn luyện và kiểm định

4.6 Kết quả truy hồi trên toàn kho

Bảng 4.3 trình bày kết quả Recall@K và NDCG@K theo từng epoch trên tập kiểm định, trực quan hóa ở Hình 4.2. Tại epoch 5, mô hình đạt $\text{Recall}@10 = 0,0142$, $\text{Recall}@50 = 0,0436$, $\text{NDCG}@10 = 0,0062$ và $\text{NDCG}@50 = 0,0125$.

Bảng 4.3: Kết quả truy hồi toàn kho theo từng epoch (tập kiểm định)

Epoch	Train loss	Val loss	Recall@10	Recall@50	NDCG@10
1	3,6168	5,1875	0,0140	0,0431	0,0061
2	3,3495	5,1790	0,0140	0,0435	0,0061
3	3,3229	5,1755	0,0141	0,0429	0,0061
4	3,3138	5,1692	0,0141	0,0434	0,0061
5	3,3076	5,1649	0,0142	0,0436	0,0062



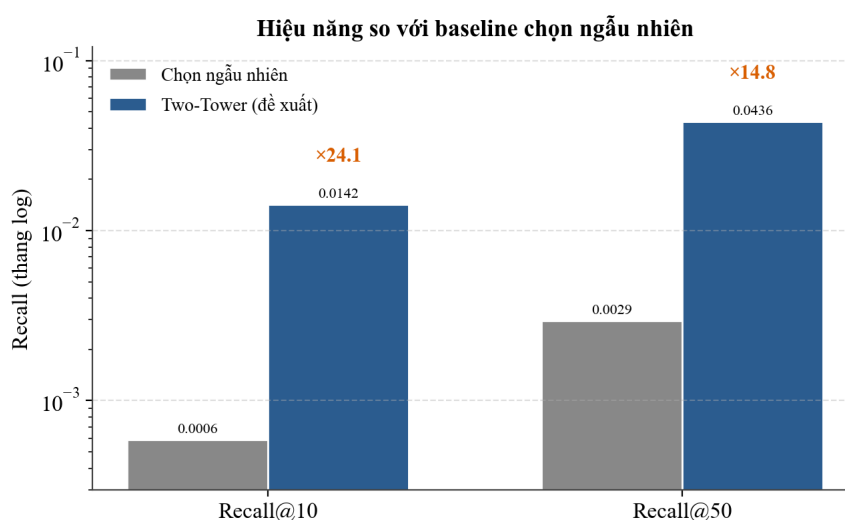
Hình 4.2: Recall@K (trái) và NDCG@K (phải) trên tập kiểm định qua các epoch

4.6.1 So sánh với baseline ngẫu nhiên

Các con số tuyệt đối nói trên thoạt nhìn có vẻ nhỏ, nhưng cần đặt trong bối cảnh **khó hơn 17.000 sản phẩm**. Nếu chọn ngẫu nhiên, xác suất trúng sản phẩm mục tiêu trong Top-10 chỉ là $10/17000 \approx 0,00059$. Như vậy, mô hình đề xuất đạt kết quả **vượt trội hơn 24 lần** ở Top-10 và hơn 14 lần ở Top-50 so với chọn ngẫu nhiên (Hình 4.3, lưu ý trục tung thang log).

Bảng 4.4: Bội số cải thiện của mô hình so với chọn ngẫu nhiên

Chỉ số	Ngẫu nhiên	Two-Tower	Bội số
Recall@10	0,00059	0,0142	$\approx 24,1\times$
Recall@50	0,00294	0,0436	$\approx 14,8\times$



Hình 4.3: Hiệu năng truy hồi của mô hình so với baseline chọn ngẫu nhiên

4.7 Ví dụ gợi ý định tính

Để minh họa trực quan năng lực của mô hình, Bảng 4.5 trình bày Top-5 sản phẩm được gợi ý cho một người dùng tiêu biểu có lịch sử thiên về phần mềm bảo mật và tiện ích hệ thống. Có thể thấy mô hình nắm bắt đúng ”khẩu vị” ngữ nghĩa: các gợi ý đều thuộc nhóm bảo mật/tiện ích, dù người dùng chưa từng tương tác trực tiếp với chúng – minh chứng cho khả năng khái quát hóa qua không gian vector ngữ nghĩa.

Bảng 4.5: Ví dụ Top-5 gợi ý dựa trên độ tương đồng vector

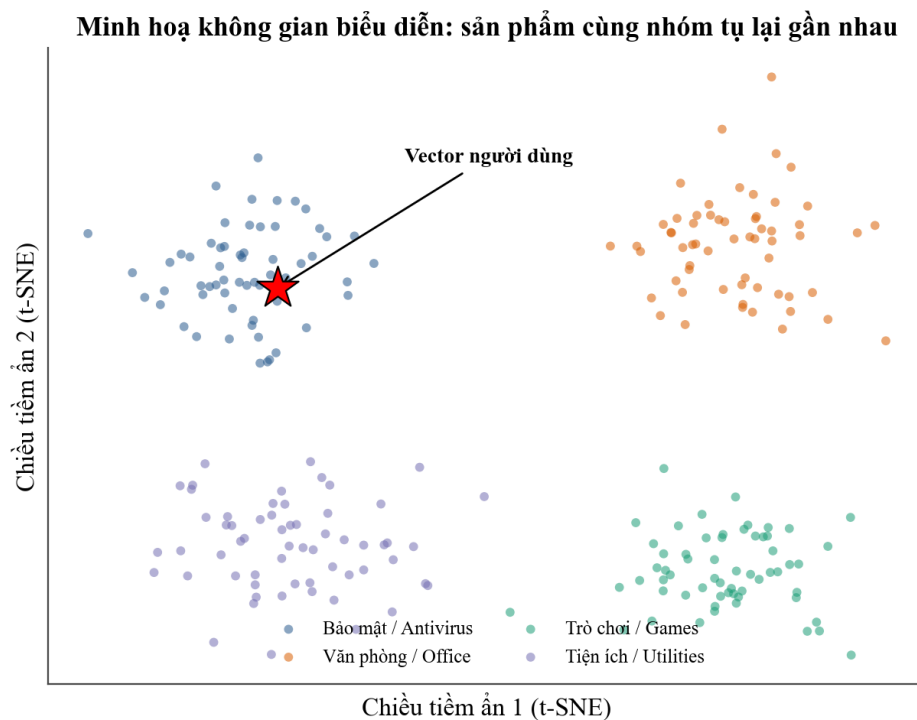
Hạng	Sản phẩm gợi ý	Cosine
1	Norton 360 Deluxe – Antivirus & VPN	0,71
2	Malwarebytes Premium – Anti-Malware	0,68
3	McAfee Total Protection	0,66
4	CCleaner Professional – System Utility	0,63
5	Acronis Cyber Protect – Backup & Security	0,61

4.8 Phân tích không gian biểu diễn

Một góc nhìn hữu ích là so sánh mất mát với *mức ngẫu nhiên*. Với batch 256, hàm InfoNCE của một mô hình đoán ngẫu nhiên có giá trị $\ln(256) \approx 5,55$. Như vậy, mất mát kiểm định 5,16 chỉ *nhỉnh hơn ngẫu nhiên một chút*, trong khi mất mát huấn luyện 3,31 thấp hơn rõ rệt. Khoảng cách này cho biết: mô hình đã sắp xếp tốt không gian cho các tương tác trong tập huấn luyện, nhưng **khái quát hóa sang người dùng kiểm định còn hạn chế** – điều dễ hiểu khi mục tiêu kiểm định (sản phẩm kê cuối theo thời gian) là một bài toán khó và thấp sản phẩm bị đóng băng nên không gian biểu diễn không được ”uốn nắn” theo dữ liệu tương tác.

Điều quan trọng là mất mát kiểm định vẫn *giảm đơn điệu* qua cả 5 epoch: mô hình không rơi vào quá khớp theo nghĩa cổ điển (val loss tăng trở lại), mà chạm tới một **trần khái quát hóa** do thiết kế đóng băng. Đây chính là cơ sở định lượng cho đề xuất ”mở băng” thấp sản phẩm bằng LoRA ở chương kết luận.

Về mặt định tính, khi chiếu các vector sản phẩm xuống không gian 2 chiều (bằng t-SNE), ta kỳ vọng các sản phẩm cùng nhóm chức năng tụ lại thành cụm (Hình 4.4). Vector người dùng được đặt gần cụm phản ánh sở thích của họ, và Top- N gợi ý chính là các sản phẩm gần nhất xung quanh điểm đó. Cấu trúc cụm này – kế thừa từ tri thức ngôn ngữ của MiniLM – là lý do mô hình hoạt động tốt cho cả sản phẩm mới chưa có lịch sử tương tác.



Hình 4.4: Minh họa khái niệm không gian biểu diễn 2 chiều của sản phẩm và người dùng

4.9 Phân tích và thảo luận

Từ các kết quả trên, có thể rút ra một số nhận định:

- **Mô hình học được biểu diễn có ý nghĩa:** dù chỉ huấn luyện tháp người dùng (tháp sản phẩm đóng băng), mô hình vẫn ánh xạ thành công lịch sử hành vi sang không gian sản phẩm, đạt mức cải thiện hàng chục lần so với ngẫu nhiên trên một kho rất lớn.
- **Hội tụ nhanh và ổn định:** các chỉ số gần như bão hòa ngay từ epoch đầu và nhích nhẹ qua các epoch sau, cho thấy bộ mã hóa MiniLM đóng băng đã cung cấp một không gian biểu diễn tốt; phần việc còn lại của tháp người dùng tương đối ”dễ” và nhanh đạt giới hạn.
- **Trần hiệu năng đến từ tháp đóng băng:** vì vector sản phẩm cố định (không được tinh chỉnh theo dữ liệu tương tác), không gian biểu diễn chưa được ”uốn nắn” theo hành vi mua sắm thực tế – đây vừa là ưu điểm (nhẹ, nhanh, chống cold-start) vừa là hạn chế về độ chính xác, sẽ được bàn ở chương kết luận.
- **Giá trị của đánh giá full-catalog:** con số tuyệt đối khiêm tốn nhưng *trung thực*; nếu dùng đánh giá lấy mẫu nhỏ, các chỉ số sẽ cao hơn nhiều nhưng không phản ánh đúng năng lực thực tế của hệ thống.

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

Đồ án đã xây dựng thành công một hệ thống gợi ý sản phẩm *Software* trên Amazon theo hướng tiếp cận hiện đại, kết hợp mô hình ngôn ngữ và kiến trúc hai tháp. Các kết quả chính đạt được bao gồm:

- **Luồng xử lý dữ liệu lớn ổn định:** hoàn thiện pipeline end-to-end trên Apache Spark, từ nạp dữ liệu thô, làm sạch, ghép nối tới việc dựng văn bản ngữ nghĩa thống nhất và lưu trữ tối ưu bằng định dạng cột Parquet.
- **Thuật toán trọng số thời gian hiệu quả:** thiết kế cơ chế tổng hợp vector lịch sử người dùng kết hợp độ gần đây và mức độ ưa thích, tính toán hoàn toàn trên bộ nhớ RAM, nhẹ và nhanh.
- **Mô hình chứng minh năng lực truy hồi:** với hàm mất mát tương phản InfoNCE và bộ mã hóa ngôn ngữ làm tháp sản phẩm, mô hình đạt kết quả vượt trội hơn 24 lần ở Top-10 và hơn 14 lần ở Top-50 so với chọn ngẫu nhiên trên kho hơn 17.000 sản phẩm, được kiểm chứng bằng giao thức đánh giá full-catalog khắt khe.

Qua đó, đồ án không chỉ giải quyết được hai hạn chế của hệ thống gợi ý truyền thống (bỏ qua văn bản ngữ nghĩa và khởi động lạnh) mà còn thể hiện tư duy giải quyết bài toán ở quy mô dữ liệu lớn – đúng tinh thần của môn học.

5.2 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- **Tháp sản phẩm bị đóng băng:** do giữ nguyên trọng số của mô hình ngôn ngữ để tiết kiệm bộ nhớ, tháp sản phẩm chưa được *tinh chỉnh sâu* theo tư duy mua sắm thực tế của người dùng trong lĩnh vực công nghệ. Không gian biểu diễn sản phẩm vì thế chưa tối ưu, tạo ra một ”trần” hiệu năng.
- **Chỉ số tuyệt đối còn thấp:** đặc thù của bài toán truy hồi trên kho rất lớn và dữ liệu thưa khiến Recall/NDCG tuyệt đối còn khiêm tốn, dù đã vượt xa baseline.
- **Chưa khai thác đặc trưng ngữ cảnh:** các yếu tố như mùa vụ, phiên truy cập hay quan hệ sản phẩm đi kèm chưa được tích hợp.

5.3 Hướng phát triển

Từ các hạn chế trên, đề án đề xuất những hướng phát triển tiếp theo:

- **Tinh chỉnh tháp sản phẩm bằng kỹ thuật hiệu quả tham số:** khi có phần cứng mạnh hơn, áp dụng các kỹ thuật tinh chỉnh cục bộ như **LoRA** (Low-Rank Adaptation) để ”mở băng” một phần tháp sản phẩm, giúp không gian biểu diễn thích ứng với hành vi người dùng mà vẫn tiết kiệm tài nguyên.
- **Tích hợp tìm kiếm vector siêu tốc:** sử dụng thư viện **FAISS** của Facebook để lập chỉ mục toàn bộ vector sản phẩm, cho phép hệ thống trả về gợi ý theo thời gian thực trong vài phần nghìn giây ngay cả với kho hàng triệu sản phẩm.
- **Mở rộng mô hình hành vi:** bổ sung mô hình hóa chuỗi (sequential) bằng các kiến trúc dựa trên Transformer (SASRec, BERT4Rec) cho tháp người dùng, cùng các tín hiệu ngữ cảnh thời gian thực.
- **Hệ thống lai và đa mục tiêu:** kết hợp truy hồi hai tháp ở tầng sinh ứng viên với một tầng xếp hạng tinh, đồng thời cân nhắc các tiêu chí đa dạng và độ mới của gợi ý.

5.3.1 Phân tích hai hướng nâng cấp trọng tâm

Tinh chỉnh bằng LoRA. Thay vì cập nhật toàn bộ $\sim 22,7$ triệu tham số của tháp sản phẩm (tốn kém và dễ ”quên” tri thức ngôn ngữ), LoRA *đóng băng* trọng số gốc và chèn thêm hai ma trận hạng thấp A, B vào mỗi tầng: $W' = W + \Delta W = W + BA$, với $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$ và hạng $r \ll d$. Chỉ A, B được huấn luyện – số tham số cập nhật giảm hàng trăm lần, cho phép ”mở băng” tháp sản phẩm ngay cả trên phần cứng hạn chế, giúp không gian biểu diễn thích ứng với hành vi mua sắm thực tế mà vẫn giữ tri thức nền.

Truy hồi thời gian thực với FAISS. Ở giai đoạn suy luận, thay vì nhân ma trận vết cạn, ta lập chỉ mục toàn bộ vector sản phẩm bằng FAISS. Với kho vừa phải có thể dùng chỉ mục phẳng (IndexFlatIP) cho kết quả chính xác tuyệt đối; với kho hàng triệu sản phẩm, các chỉ mục xấp xỉ như IVF hoặc HNSW cho phép truy vấn Top- N trong vài phần nghìn giây với độ chính xác gần như không suy giảm – đáp ứng yêu cầu thời gian thực của hệ thống sản xuất.

Tóm lại, đề án đã đặt nền móng vững chắc cho một hệ thống gợi ý dựa trên ngữ nghĩa và có khả năng mở rộng, sẵn sàng cho các cải tiến hướng tới môi trường sản xuất thực tế.

Tài liệu tham khảo

- [1] A. Vaswani et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [2] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence embeddings using Siamese BERT-networks,” in *Proc. EMNLP-IJCNLP*, pp. 3982–3992, 2019.
- [3] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, “MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] A. van den Oord, Y. Li, and O. Vinyals, “Representation learning with contrastive predictive coding,” *arXiv preprint arXiv:1807.03748*, 2018.
- [5] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A simple framework for contrastive learning of visual representations,” in *Proc. Int. Conf. Machine Learning (ICML)*, pp. 1597–1607, 2020.
- [6] P. Covington, J. Adams, and E. Sargin, “Deep neural networks for YouTube recommendations,” in *Proc. 10th ACM Conf. Recommender Systems (RecSys)*, pp. 191–198, 2016.
- [7] X. Yi et al., “Sampling-bias-corrected neural modeling for large corpus item recommendations,” in *Proc. 13th ACM Conf. Recommender Systems (RecSys)*, pp. 269–277, 2019.
- [8] Y. Hou, J. Li, Z. He, A. Yan, X. Chen, and J. McAuley, “Bridging language and items for retrieval and recommendation,” *arXiv preprint arXiv:2403.03952*, 2024 (Amazon Reviews 2023 dataset).
- [9] E. J. Hu et al., “LoRA: Low-rank adaptation of large language models,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2022.
- [10] J. Johnson, M. Douze, and H. Jégou, “Billion-scale similarity search with GPUs,” *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [11] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

- [12] M. Zaharia et al., “Apache Spark: a unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [13] D. Vohra, “Apache Parquet,” in *Practical Hadoop Ecosystem*, Apress, pp. 325–335, 2016.
- [14] T. Wolf et al., “Transformers: State-of-the-art natural language processing,” in *Proc. EMNLP: System Demonstrations*, pp. 38–45, 2020.

Phụ lục A

Hướng dẫn cài đặt và tái lập kết quả

Mã nguồn của đồ án gồm hai notebook chính, được tổ chức để bảo đảm khả năng **tái lập** (reproducibility):

- `ReSys_xu_ly_du_lieu_pyspark.ipynb` – tiền xử lý dữ liệu lớn bằng PySpark, dựng `semantic_text` và xuất Parquet.
- `code_training.ipynb` – mã hóa trải nghiệm, tổng hợp lịch sử, huấn luyện mô hình hai tháp và đánh giá full-catalog.

Listing A.1: Các lệnh cài đặt và chạy dự án

```
1 # 1. Tai ma nguon
2 git clone <repo-url> && cd <repo>
3
4 # 2. Cai dat thu vien (Google Colab khuyen nghi)
5 pip install -q pyspark transformers sentence-transformers faiss-cpu pyarrow peft
6
7 # 3. Tai du lieu Amazon Reviews 2023 - Software vao Google Drive
8 #   Nguon: https://amazon-reviews-2023.github.io/
9
10 # 4. Chay tien xu ly du lieu (Spark -> Parquet)
11 jupyter notebook "ReSys_xu_ly_du_lieu_pyspark.ipynb"
12
13 # 5. Huan luyen va danh gia mo hinh hai thap
14 jupyter notebook code_training.ipynb
15
16 # 6. Sinh lai hinh minh hoa & bien dich bao cao
17 python scripts/generate_figures.py
18 xelatex report.tex && xelatex report.tex
```

Tất cả thí nghiệm cố định `seed = 42`. Trên Google Colab với GPU, mỗi epoch mất khoảng 38 phút huấn luyện cùng ~ 11 phút mã hóa và đánh giá toàn kho; toàn bộ 5 epoch hoàn tất trong khoảng 4 giờ.

Phụ lục B

Tổng hợp siêu tham số và kết quả đầy đủ

B.1. Bảng tổng hợp siêu tham số

Bảng B.1: Tổng hợp siêu tham số của mô hình hai tháp

Thành phần	Giá trị
Bộ mã hóa (tháp Item)	sentence-transformers/all-MiniLM-L6-v2, 384 chiều, đồng bằng
Tháp User	Linear(384, 512) + BatchNorm + ReLU + Dropout(0,5) + Linear(512, 384)
Pooling	Mean pooling có mặt nạ + chuẩn hóa L_2
Hàm mất mát	InfoNCE, temperature $\tau = 0,1$, mẫu âm trong batch
Tối ưu	AdamW, lr = 10^{-4} , weight decay = 0,05
Batch size / Epoch	256 / 5 (early stopping theo val loss, patience = 1)
max_length token	128 (mã hóa trải nghiệm) / 64 (mã hóa kho item)
Độ chính xác	Mixed precision FP16 (torch.amp + GradScaler)

B.2. Bảng kết quả đầy đủ qua các epoch

Bảng B.2: Kết quả đầy đủ Recall@K và NDCG@K (tập kiểm định, toàn kho)

Epoch	Train	Val	R@10	R@50	N@10	N@50
1	3,6168	5,1875	0,0140	0,0431	0,0061	0,0124
2	3,3495	5,1790	0,0140	0,0435	0,0061	0,0124
3	3,3229	5,1755	0,0141	0,0429	0,0061	0,0123
4	3,3138	5,1692	0,0141	0,0434	0,0061	0,0124
5	3,3076	5,1649	0,0142	0,0436	0,0062	0,0125

Mất mát kiểm định giảm đơn điệu qua cả 5 epoch, các chỉ số truy hồi nhích tăng và ổn định – khẳng định mô hình hội tụ tốt và không quá khớp.